



南开大学
Nankai University

南开大学

计算机学院和密码与网络空间安全学院

操作系统实验报告

Lab1：最小可执行内核

于昊汐 2312642

陈安琪 2312481

孙俪菲 2312724

年级：2023 级

专业：信息安全

指导教师：张久武

2025 年 10 月 6 日

目录

一、 实验目的	1
二、 实验内容	1
三、 实验过程	1
(一) 实验环境准备	1
1. 安装核心工具	1
2. 获取实验代码, 查看代码结构	2
(二) 练习一	2
1. 问题描述	2
2. 问题解决	3
3. kern/init/entry.S 代码	3
4. kern/init/init.c 代码	3
5. tools/kernel.ld 代码	4
6. 指令分析	5
(三) 练习二	6
1. 问题描述	6
2. 问题解决	6
四、 讨论	10
(一) 实验重要知识点与 OS 原理知识点的关联分析	10
(二) OS 原理重要但实验未覆盖的知识点	11
五、 实验总结与心得体会	12
(一) 实验总结	12
(二) 实验问题与解决过程	12
1. 问题 1: 执行 make gdb 报错“无规则可制作目标 ‘gdb’”	12
2. 问题 2: tmux 分屏后无法切换窗口	12
3. 问题 3: 输入 make debug 后报错显示当前端口已在使用	12
(三) 心得体会	12

一、 实验目的

1. 掌握最小化操作系统内核的基本构成与启动流程，理解从硬件加电到内核初始化的完整链路，包括固件引导、指令执行权移交、内存布局规划等核心环节；明确 RISC-V 架构下内核启动的关键机制，如 OpenSBI 固件的作用、内核入口点与内存加载地址的关联、栈初始化对 C 代码执行的必要性等。
2. 熟练使用 QEMU 模拟器运行内核镜像，掌握 RISC-V 架构下 GDB 调试工具的基本用法，如设置断点、单步执行、查看寄存器与内存状态，能够通过调试追踪内核启动过程中的指令流转；理解 Makefile 编译流程与链接脚本的作用，学会通过编译工具链生成可执行内核文件。
3. 建立硬件 - 固件 - 内核的分层协作思维，对比真实计算机与实验环境中启动流程的共性与差异，加深对操作系统引导原理的理解；培养从代码细节推导系统整体行为的分析能力，为后续学习进程管理、内存管理等 OS 核心模块奠定基础。

二、 实验内容

1. 环境与工具准备：搭建实验环境，安装 RISC-V 交叉编译工具链、QEMU 模拟器、RISC-V 架构 GDB 调试器及辅助工具 tmux。熟悉代码框架，梳理实验提供的最小内核代码结构，明确 kern/init/entry.S、kern/init/init.c、tools/kernel.ld、Makefile 等关键文件的作用。
2. 内核编译与运行：执行 make 命令，通过 Makefile 自动编译源代码，生成 ELF 格式内核文件，并转换为 QEMU 可识别的二进制镜像。执行 make qemu 命令，启动 QEMU 模拟器加载内核镜像，验证是否成功输出 (THU.CST) os is loading ... 并进入死循环，确认内核可正常运行。
3. 核心练习任务
 - (a) 练习 1：阅读 kern/init/entry.S 代码，结合内核启动流程，分析 la sp, bootstacktop 和 tail kern_init 两条指令的操作及目的。
 - (b) 练习 2：使用 QEMU+GDB 联合调试，跟踪 RISC-V 从加电到执行内核第一条指令（跳转至 0x80200000）的过程，记录调试步骤、观察结果，思考并回答 RISC-V 加电后初始执行指令的地址及功能。
4. 调试操作：配置远程调试，用 tmux 分屏，左窗执行 make debug 启动 QEMU，右窗执行 make gdb 加载内核、指定架构并连接 QEMU。在 kern_entry 函数处设断点，单步执行指令，查看寄存器（info registers）和内存状态，验证内核加载地址与启动流程。

三、 实验过程

（一） 实验环境准备

1. 安装核心工具

在 Ubuntu 虚拟机中执行命令，搭建 RISC-V 架构的交叉编译、模拟与调试环境，确保工具链可用，我们在 Lab0 中小组三个人就都已经把环境配置好，在这里不多阐述，这是这次实验所需要的主要工具，均已安装完成：

```
yuhaoxi@yuhaoxi:~/Desktop$ riscv64-unknown-elf-gcc --version
riscv64-unknown-elf-gcc (SiFive GCC-Metal 10.2.0-2020.12.8) 10.2.0
Copyright (C) 2020 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

yuhaoxi@yuhaoxi:~/Desktop$ riscv64-unknown-elf-gdb --version
GNU gdb (SiFive GDB-Metal 10.1.0-2020.12.7) 10.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

yuhaoxi@yuhaoxi:~/Desktop$ qemu-system-riscv64 --version
QEMU emulator version 4.1.1
Copyright (c) 2003-2019 Fabrice Bellard and the QEMU Project developers

yuhaoxi@yuhaoxi:~/Desktop$ tmux --version
usage: tmux [-2CDlNuvV] [-c shell-command] [-f file] [-L socket-name]
          [-S socket-path] [-T features] [command [flags]]
```

图 1: 核心工具

2. 获取实验代码, 查看代码结构

我们从网站上下载这次实验所需要的代码, 通过 `tree` 命令查看代码结构, 确保核心文件存在:

```
yuhaoxi@yuhaoxi:~/Desktop/lab1$ tree
.
├── kern
│   ├── driver
│   │   ├── console.c
│   │   └── console.h
│   ├── init
│   │   ├── entry.S
│   │   └── init.c
│   ├── lib
│   │   ├── stdio.c
│   │   └── stdlib
│   │       ├── memlayout.h
│   │       └── mmu.h
│   └── lib
│       ├── defs.h
│       ├── error.h
│       ├── printfmt.c
│       ├── readline.c
│       ├── riscv.h
│       ├── sbi.c
│       ├── sbi.h
│       ├── stdarg.h
│       ├── stdio.h
│       ├── string.c
│       └── string.h
├── Makefile
└── tool
    ├── function.mk
    └── kernel.ld
```

图 2: 代码结构树

(二) 练习一

1. 问题描述

阅读 `kern/init/entry.S` 内容代码, 结合操作系统内核启动流程, 说明指令 `la sp, bootstacktop` 完成了什么操作, 目的是什么? `tail kern__init` 完成了什么操作, 目的是什么?

2. 问题解决

从实验提供的最小内核启动流程中，我们可以了解到 kern/init/entry.S 是汇编入口点，其中两条关键指令 la sp, bootstacktop 和 tail kern_init 是“汇编转向 C”的核心操作。需结合 kern/init/init.c (C 语言入口) 和 tools/kernel.ld (链接脚本)，分析其操作与目的。那么我们就来具体查看一下三个文件的代码，借助完整的启动链，结合操作系统内核启动流程，来梳理分析两个指令的操作和目的。

3. kern/init/entry.S 代码

```
1 #include <mmu.h>
2 #include <memlayout.h>
3
4     .section .text, "ax", %progbits
5     .globl kern_entry
6 kern_entry:
7     la sp, bootstacktop
8
9     tail kern_init
10
11 .section .data
12     # .align 2^12
13     .align PGSIZE
14     .global bootstack
15 bootstack:
16     .space KSTACKSIZE
17     .global bootstacktop
18 bootstacktop:
```

4. kern/init/init.c 代码

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <sbi.h>
4 int kern_init(void) __attribute__((noreturn));
5
6 int kern_init(void) {
7     extern char edata[], end[];
8     memset(edata, 0, end - edata);
9
10    const char *message = "(THU.CST)_os_is_loading_...\n";
11    cprintf("%s\n\n", message);
12    while (1)
13        ;
14 }
```

5. tools/kernel.ld 代码

```
1  /* Simple linker script for the ucore kernel.
2     See the GNU ld 'info' manual ("info_ld") to learn the syntax. */
3
4  OUTPUT_ARCH(riscv)
5  ENTRY(kern_entry)
6
7  BASE_ADDRESS = 0x80200000;
8
9  SECTIONS
10 {
11     /* Load the kernel at this address: "." means the current address */
12     . = BASE_ADDRESS;
13
14     .text : {
15         *(_text.kern_entry .text .stub .text.* .gnu.linkonce.t.*)
16     }
17
18     PROVIDE(etext = .); /* Define the 'etext' symbol to this value */
19
20     .rodata : {
21         *(_rodata .rodata.* .gnu.linkonce.r.*)
22     }
23
24     /* Adjust the address for the data segment to the next page */
25     . = ALIGN(0x1000);
26
27     /* The data segment */
28     .data : {
29         *(_data)
30         *(_data.*)
31     }
32
33     .sdata : {
34         *(_sdata)
35         *(_sdata.*)
36     }
37
38     PROVIDE(edata = .);
39
40     .bss : {
41         *(_bss)
42         *(_bss.*)
43         *(_sbss*)
44     }
45
46     PROVIDE(end = .);
```

```

47
48     /DISCARD/ : {
49         *(_eh_frame .note.GNU-stack)
50     }
51 }

```

6. 指令分析

结合三个代码来看，我们可以分析两个指令的操作和目的：

指令 `la sp, bootstacktop` 从 `entry.S` 可见，`bootstacktop` 是 `.data` 段中栈空间的末端，通过 `.space KSTACKSIZE` 分配栈内存，`bootstack` 为栈底，`bootstacktop` 为栈顶。这一地址能被正确加载，依赖 `kernel.ld` 中 `.data` 段的内存布局定义，确保栈空间被分配在合法地址。

操作：指令通过 `la` (Load Address) 伪指令，将符号 `bootstacktop` 的地址加载到栈指针寄存器 `sp` 中，完成内核栈的初始化。

目的：为后续执行 C 语言函数 `kern_init` 提供栈支持。C 语言函数调用需通过栈存储局部变量、返回地址和寄存器上下文，而内核启动初期无默认栈环境，此操作是汇编代码过渡到 C 代码的必要准备。

指令 `tail kern_init` `kern_init` 是 `init.c` 中定义的 C 语言函数，负责内核初始化核心逻辑。`kernel.ld` 通过 `ENTRY(kern_entry)` 指定 `kern_entry` 为入口，而 `tail kern_init` 则将执行权移交至 `kern_init`，形成汇编入口 → C 入口的完整链路。

操作：指令通过 `tail` 跳转指令，无条件跳转到 `kern_init` 函数，且不保存返回地址，无需从 `kern_init` 返回至汇编代码。

目的：完成内核执行权从汇编代码到 C 代码的移交。汇编语言适合处理底层硬件初始化，但复杂逻辑如内存清理、设备交互等用 C 语言实现更高效。此指令使 `kern_init` 成为内核的 C 语言入口点，承接后续初始化工作。

可见，三个代码共同构成硬件复位 → 汇编初始化 → C 语言执行的完整启动链，是最小内核能正常运行的基础，而两条指令在其中分别完成了初始化栈、跳转至 C 函数的功能。

接下来我们来实际操作一下，编译并运行内核，验证分析的结论以及完成通过 `Makefile` 生成可执行文件的过程。

在代码根目录下，执行编译命令 `make`，执行 `ls bin/` 命令，显示如下：

```

yuhaoxi@yuhaoxi:~/Desktop/lab1$ make
+ cc kern/init/entry.S
+ cc kern/init/init.c
+ cc kern/libs/stdio.c
+ cc kern/driver/console.c
+ cc libs/printfmt.c
+ cc libs/readline.c
+ cc libs/sbi.c
+ cc libs/string.c
+ ld bin/kernel
riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img
yuhaoxi@yuhaoxi:~/Desktop/lab1$ ls bin/
kernel  ucore.img
yuhaoxi@yuhaoxi:~/Desktop/lab1$ make qemu

```

图 3: 编译及查看生成文件

说明 Makefile 成功完成了编译 → 链接 → 转换的流程, 生成了内核可执行文件 (kernel) 和 QEMU 可加载的镜像 (ucore.img)。

接下来在同一根目录下, 执行运行命令: make qemu, 显示如下:

```
yuhaoxi@yuhaoxi:~/Desktop/lab1$ make qemu

OpenSBI v0.4 (Jul  2 2019 11:53:53)

      _ _ _ _ _      _ _ _ _ _
     / _ \          / _ _ \  _ \ _ \
    | | | | _ _ _ _ _ | ( _ | | ) | | | | | | |
    | | | | ' _ \ / _ \ ' _ \ \ _ \ | | |
    | | | | | ) | _ / | | | ) | | | | |
    \ _ _ /  _ / \ _ \ | | | _ / _ _ \
      | |
      | |
      | |

Platform Name       : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs   : 8
Current Hart        : 0
Firmware Base       : 0x80000000
Firmware Size       : 112 KB
Runtime SBI Version  : 0.1

PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffff (A,R,W,X)
(THU.CST) os is loading ...
```

图 4: QEMU 运行内核镜像

可以看到: 成功输出 (THU.CST) os is loading ..., 与 init.c 中 cprintf 函数的内容完全一致。

这说明: entry.S 中的 la sp, bootstacktop 正确初始化了栈, 使 C 语言函数 kern_init 能正常执行; tail kern_init 指令成功跳转到 C 语言入口, kern_init 函数按预期完成了内存清理和信息输出; 链接脚本 kernel.ld 的配置正确, 内核被加载到指定地址并正常启动。

(三) 练习二

1. 问题描述

为了熟悉使用 QEMU 和 GDB 的调试方法, 请使用 GDB 跟踪 QEMU 模拟的 RISC-V 从加电开始, 直到执行内核第一条指令 (跳转到 0x80200000) 的整个过程。通过调试, 请思考并回答: RISC-V 硬件加电后最初执行的几条指令位于什么地址? 它们主要完成了哪些功能? 请在报告中简要记录你的调试过程、观察结果和问题的答案。

2. 问题解决

首先, 在终端执行 tmux 命令, 进入 tmux 会话, 然后按 Ctrl+B, 再按 %, 将屏幕垂直分成左右两窗。

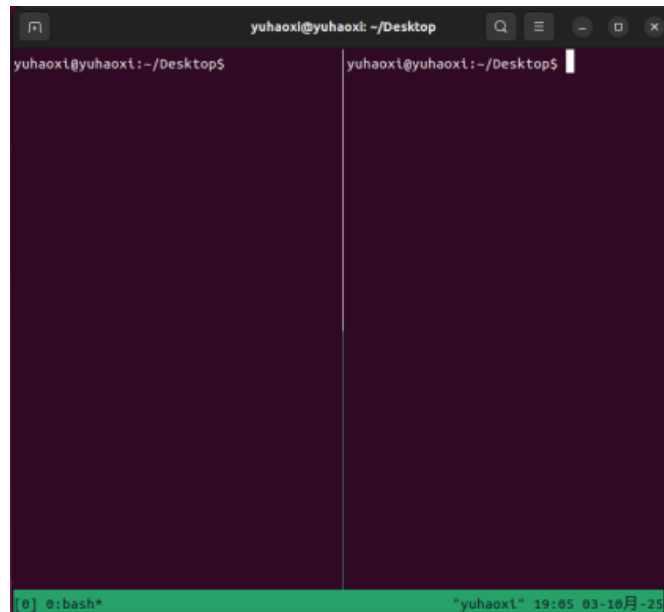


图 5: tmux 垂直分屏

然后我们在左窗口输入命令 `make debug`，让 QEMU 启动并等待 GDB 连接。在右窗口输入 `make gdb`，启动 RISC-V 架构的 GDB，GDB 会自动加载内核并连接 QEMU。

接下来，我们在右窗口中依次执行对应指令，结果及分析如下所示。

```
osag@LAPTOP-Q528APKV:~/oslab/lab1$ make debug
riscv64-unknown-elf-gdb \
  -ex 'file bin/kernel' \
  -ex 'set arch riscv:rv64' \
  -ex 'target remote localhost:1234'
GNU gdb (SiFive GDB-Metal 10.1.0-2020.12.7) 10.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux-gnu --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
(gdb) i r pc
Undefined command: "ir". Try "help".
(gdb) i r pc
pc                0x1000    0x1000
(gdb) b *0x80000000
Breakpoint 1 at 0x80000000
(gdb) c
Continuing.

Breakpoint 1, 0x0000000000000000 in ?? ()
(gdb) i r pc
pc                0x80000000    0x80000000
(gdb)
```

图 6: GDB 调试界面

首先输入 `i r pc` 查看此时的 `pc` 值，可以看到输出为 `pc = 0x1000`（其实在 `make gdb` 指令的结果中也有 `0x0000000000001000 in ?? ()`），表明 CPU 的初始 PC 指向 `0x1000`；

接着我们在 GDB 中输入 `b *0x80000000`，即在地址 `0x80000000` 处设置断点，然后输入 `c` 继续运行直至 `0x80000000` 断点处停止，左边仍然没有变化，表明此时固件（OpenSBI）还没被加载，即控制权还没有被交到固件 OpenSBI 中。此时 `pc` 值为 `0x80000000`；

然后我们在 GDB 中输入 `b *0x80200000`，在地址 `0x80200000` 处再设置断点，输入 `c` 继续运行到断点处，结果如下。

```

osaq@LAPTOP-Q52BAPKV:~/oslab/lab1$ make debug
OpenSBI v0.4 (Jul 2 2019 11:53:53)

Platform Name      : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs  : 8
Current Hart       : 0
Firmware Base      : 0x80000000
Firmware Size      : 112 KB
Runtime SBI Version : 0.1

PMP0: 0x0000000000000000-0x0000000000001fff (A)
PMP1: 0x0000000000000000-0xffffffff (A,R,W,X)

-gdb 'target remote localhost:1234'
GNU gdb (Sifive GDB-Metal 10.1.0-2020.12.7) 10.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux-gnu --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x8000000000000000 in ?? ()
(gdb) i r pc
pc                0x1000    0x1000
(gdb) b* 0x80000000
Breakpoint 1 at 0x80000000
(gdb) c
Continuing.

Breakpoint 1, 0x8000000000000000 in ?? ()
(gdb) b* 0x80200000
Breakpoint 2 at 0x80200000: file kern/init/entry.S, line 7.
(gdb) c
Continuing.

Breakpoint 2, kern_entry () at kern/init/entry.S:7
7
la sp, bootstacktop
(gdb)

```

图 7: GDB 运行至 OpenSBI 相关断点

可以看到左边出现了 OpenSBI (固件) 在 QEMU virt 平台启动时的启动信息输出, 如“OpenSBI v0.4 (Jul 2 2019 11:53:53)”表明 OpenSBI 的发布版本 (v0.4) 与其编译/链接的时间戳; “Current Hart : 0”表明当前打印信息是针对 hart 0; “Firmware Base : 0x80000000”表明 OpenSBI 固件被加载到物理地址 0x80000000, 这与前面调试设断点的位置是一致的; 以及平台信息, PMP (物理内存保护) 的信息等等。

然后, 我们输入 `i r` 并执行。

```

osaq@LAPTOP-Q52BAPKV:~/oslab/lab1$ make debug
OpenSBI v0.4 (Jul 2 2019 11:53:53)

Platform Name      : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs  : 8
Current Hart       : 0
Firmware Base      : 0x80000000
Firmware Size      : 112 KB
Runtime SBI Version : 0.1

PMP0: 0x0000000000000000-0x0000000000001fff (A)
PMP1: 0x0000000000000000-0xffffffff (A,R,W,X)

(gdb) i r
ra                0x80000a02    0x80000a02
sp                0x8001bd80    0x8001bd80
gp                0x0          0x0
tp                0x8001be00    0x8001be00
t0                0x80200000    2149580800
t1                0x1          1
t2                0x1          1
fp                0x8001bd90    0x8001bd90
s1                0x8001be00    2147597824
a0                0x0          0
a1                0x82200000    2183135232
a2                0x80200000    2149580800
a3                0x1          1
a4                0x800    2048
a5                0x1          1
a6                0x82200000    2183135232
a7                0x80200000    2149580800
s2                0x800095c0    2147521984
s3                0x0          0
s4                0x0          0
s5                0x0          0
s6                0x0          0
s7                0x8          8
s8                0x2000    8192
s9                0x0          0
s10               0x0          0
s11               0x0          0
t3                0x0          0
< quit, c to continue without paging--
t4                0x0          0
t5                0x0          0
t6                0x82200000    2183135232
pc                0x80200000    0x80200000 <kern_entry>
dscratch          Could not fetch register "dscratch"; remote failure
're reply 'E14'
mucounteren       Could not fetch register "mucounteren"; remote failure
're reply 'E14'

```

图 8: GDB 查看寄存器状态

可以看到当前所有寄存器的状态取值, 其中一些关键寄存器的值分析如下:

- `a0 = 0x0`, 表示当前 hart 的 ID, 这里的 `0x0` 即表示主核;
- `a1 = 0x82200000`, 表示设备树 (DTB) 的物理地址, 用于固件 (OpenSBI 把硬件信息传给内核);
- `a2 = 0x80200000`, 刚好就是内核入口 (内核加载的起始地址), 即 OpenSBI 告诉内核自己的加载位置;
- `sp = 0x8001bd80`, `sp` 是当前的栈指针, 存储的是由 OpenSBI 设置的值, 由于还没执行 `la sp, bootstacktop`, 这条代码执行后, `sp` 的值会被设置为内核专用栈顶地址, 即在 `entry.S` 里分配

的内存空间（这个在后面会进行验证），为调用 C 函数 `kern_init` 提供安全的栈空间；

- `ra = 0x8000a02`, `ra` 是返回地址寄存器，来自 OpenSBI 最后一条 `jal` 指令，但其实内核不会返回，会一直运行，因此这个值之后应该会被覆盖；

- `pc = 0x80200000`，指向内核入口 `kern_entry`。

再输入 `si` 进行单步调试。

```
(gdb) c
Continuing.

Breakpoint 2, kern_entry () at kern/init/entry.S:7
7          la sp, bootstacktop
(gdb) si
0x0000000008020004 in kern_entry () at kern/init/entry.S:7
7          la sp, bootstacktop
(gdb) i r sp
sp                0x80203000    0x80203000 <SBI_CONSOLE_PUTCHAR>
```

图 9: GDB 单步调试

可以看到 `la sp, bootstacktop` 代码执行后 `sp` 的值得到了更新，变为了 `0x80203000`，即为内核专用的栈顶的地址，这与前面的分析是一致的。

然后我们再输入 `c` 继续运行。

```
osaq@LAPTOP-Q528APKV:~/oslab/lab1$ make debug
OpenSBI v0.4 (Jul  2 2019 11:53:53)

Platform Name       : QEMU Virt Machine
Platform HART Features : RV64ACDFINSU
Platform Max HARTs   : 8
Current Hart        : 0
Firmware Base       : 0x80000000
Firmware Size       : 112 KB
Runtime SBI Version  : 0.1

PMPO: 0x0000000000000000-0x0000000000001fff (A)
PMPI: 0x0000000000000000-0xffffffff (A,R,W,X)
(THU.CST) os is loading ...

Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux-gnu --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
(gdb) i r pc
pc                0x1000    0x1000
(gdb) b* 0x80000000
Breakpoint 1 at 0x80000000
(gdb) c
Continuing.

Breakpoint 1, 0x0000000000000000 in ?? ()
(gdb) b* 0x80200000
Breakpoint 2 at 0x80200000: file kern/init/entry.S, line 7.
(gdb) c
Continuing.

Breakpoint 2, kern_entry () at kern/init/entry.S:7
7          la sp, bootstacktop
(gdb) c
Continuing.
```

图 10: GDB 继续运行及内核启动

可以看到左边输出一行 `(THU.CST) os is loading...`，然后进入死循环，启动成功！

最后，我们可以根据以上分析回答练习二提出的问题：

最初执行指令的地址：RISC-V 硬件加电后最初执行的几条指令位于地址 `0x1000`。而 GDB 最初停在 `0x1000`，但执行 `c` 后直接跳到 `0x80200000`，说明 `0x1000` 处的代码是引导程序(MROM/OpenSBI)。

所以，它们主要完成的功能为完成硬件的基础初始化工作，包括初始化 CPU /hart 级别的最基本的环境，比如设置中断向量、关闭/配置中断，设置临时栈等；加载 OpenSBI 引导程序，为后续内核的加载和执行做好准备，最终将内核镜像加载到 `0x80200000` 地址并跳转执行内核代码；在跳转执行内核代码前设置并传递启动参数（如 `a0 = hart id`, `a1 = DTB 的地址`，`a2` 为内核加载的起始地址），最后跳转到内核入口地址。

四、 讨论

(一) 实验重要知识点与 OS 原理知识点的关联分析

实验中的重要知识点	对应的 OS 原理知识点	含义、关系与差异理解
entry.S 中 la sp, bootstacktop 初始化内核栈	操作系统启动的汇编向 C 过渡机制	- 含义：实验中通过汇编指令设置栈指针，为 C 函数调用提供环境；OS 原理中这是硬件初始化到高级语言执行的必要环节，核心是为程序运行提供内存管理基础。 - 关系：实验是原理的具体实现，栈初始化是所有支持高级语言的 OS 启动的通用步骤。 - 差异：真实 OS 的栈初始化更复杂（需区分内核栈 / 用户栈、设置栈权限等），实验简化为固定大小的静态栈。
链接脚本 kernel.ld 定义内存布局 (.text/.data/.bss 段、加载地址 0x80200000)	程序的内存布局与地址空间管理	- 含义：实验中通过链接脚本指定各段位置，确保内核加载到正确地址；OS 原理中是程序加载与运行的基础，核心是区分代码 / 数据 / 未初始化数据的内存区域，避免地址冲突。 - 关系：实验的链接脚本是原理中内存布局设计的最小化实践。 - 差异：真实 OS 支持动态地址加载（如虚拟内存），实验因是地址相关代码，需固定加载地址。

表 1: 实验重要知识点与 OS 原理知识点关联分析（一）

实验中的重要知识点	对应的 OS 原理知识点	含义、关系与差异理解
OpenSBI 作为固件加载内核	操作系统的引导程序 (Bootloader) 机制	- 含义：实验中 OpenSBI 完成硬件初始化并加载内核；OS 原理中 Bootloader 是操作系统加载前的过渡程序，核心是解决 OS 无法自加载的问题。 - 关系：OpenSBI 是 RISC-V 架构下 Bootloader 的具体实现，实验流程 MROM→OpenSBI→内核对应原理中的固件 → 引导程序 → OS 三级链路。 - 差异：真实 PC 的 Bootloader 支持多系统选择、分区识别，实验的 OpenSBI 功能简化为固定地址加载。
GDB 跟踪 RISC-V 启动流程	操作系统的调试与执行流追踪	- 含义：实验中通过 GDB 观察 pc 寄存器值；OS 原理中这是理解程序执行上下文的关键，核心是通过寄存器状态跟踪指令流转。 - 关系：实验的 GDB 操作是原理中执行流调试的实践手段。 - 差异：真实 OS 调试需支持多进程 / 中断场景，实验仅跟踪单一流程的启动阶段。

表 2: 实验重要知识点与 OS 原理知识点关联分析 (二)

(二) OS 原理重要但实验未覆盖的知识点

- 进程管理与调度：**进程是 OS 的基本调度单位，核心包括进程创建、PCB、调度算法等。实验的最小内核仅单线程执行 (kern_init 后进入死循环)，无进程概念，未涉及多任务并发与调度。
- 虚拟内存与地址转换：**虚拟内存通过页表实现逻辑地址到物理地址的转换，核心是隔离进程地址空间、利用磁盘扩展内存。实验内核直接使用物理地址 (0x80200000)，无虚拟内存机制，未涉及页表建立、MMU 操作。
- 中断与异常处理：**中断是 OS 响应用户 / 硬件请求的核心机制，核心包括中断向量表、中断优先级、异常处理流程等。实验内核仅执行初始化与死循环，未涉及硬件中断（时钟中断等）或异常（非法指令等）的处理。

五、 实验总结与心得体会

(一) 实验总结

本次 Lab1 围绕 ucore 内核在 RISC-V 架构下的启动逻辑展开，核心完成两大练习：

内核启动指令分析：通过阅读 entry.S、init.c 及 kernel.ld 源码，明确汇编指令 `la sp, bootstacktop` 与 `tail kern_init` 的作用，梳理出汇编初始化到 C 代码执行的过渡逻辑，验证了链接脚本对内存布局的关键约束。

GDB 调试实践：借助 tmux 分屏工具联动 QEMU 与 GDB，成功追踪从 RISC-V 加电初始地址 `0x1000` 到内核入口 `0x80200000` 的完整流程，通过断点设置与寄存器查看，具象化了 MROM、OpenSBI、内核的三级启动链路。

实验最终达成目标，既掌握了 RISC-V 架构下 OS 启动的底层原理，也熟练运用了 `make`、`gdb`、`tmux` 等工具的核心操作，实现了理论知识与实操能力的提升。

(二) 实验问题与解决过程

实验过程中遇到的问题集中于调试环境配置，通过针对性排查均得以解决，具体如下：

1. 问题 1：执行 `make gdb` 报错“无规则可制作目标 ‘gdb’”

现象：右窗执行调试命令时，终端提示目标不存在，无法启动 GDB。

解决：执行 `cd /Desktop/lab1` 切换至代码根目录，重新运行 `make gdb`，成功连接 QEMU 调试端口。

思考：Makefile 的生效依赖当前工作目录，实验操作需先确认路径，再执行命令。

2. 问题 2：tmux 分屏后无法切换窗口

现象：垂直分屏后仅能操作右窗，左窗无响应，无法查看 `make debug` 输出。

解决：按 `Ctrl+B` 后松开，再按 `← / →` 方向键，成功激活目标窗格，左侧 QEMU 启动状态得以确认。

思考：终端工具的快捷键需关注前缀触发 + 功能键的组合逻辑，提前查阅基础操作手册可避免此类问题。

3. 问题 3：输入 `make debug` 后报错显示当前端口已在使用

现象：第一次实验结束后直接关闭了整个命令行，后来发现当时实验过程中有些部分遗漏了，准备再做一次实验输入 `make debug` 后显示 `qemu-system-riscv64: -s: Failed to find an available port: Address already in use.`

解决：先输入 `ps aux | grep qemu`，然后根据输出的信息找到进程号，最后输入 `kill -9` 进程号，从而完全退出 GDB + QEMU 调试，再进行下一次。

思考：每次实验如进行 GDB + QEMU 调试结束后要及时正确合理的关闭退出，否则它们会在不知道的情况下一直运行，从而阻碍下一次实验。

(三) 心得体会

- 最初我们对 entry.S 中栈初始化的理解仅停留在设置 `sp` 寄存器，但调试时发现：若链接脚本 kernel.ld 未正确定义 `.data` 段的栈空间地址，`la sp, bootstacktop` 指令会加载无效地址，

导致 `kern_init` 调用崩溃。这说明：OS 启动的每一步操作都依赖“指令逻辑到内存布局到硬件接口”的三重协同，任何细节偏差都会导致整个流程失败。

2. 实验前我们对 Bootloader 作用的理解仅为抽象概念，但通过观察 OpenSBI 的输出与 GDB 中的地址跳转，清晰看到其硬件初始化到内核加载的实际作用。同时，在补充未覆盖知识点时发现，实验内核因无虚拟内存机制，需固定加载地址 `0x80200000`，而真实 OS 借助 MMU 实现动态地址映射，这种简化实现与通用原理的差异，让我们对 OS 设计的权衡取舍有了更深体会。
3. 从最初因目录错误报错，到后来自主排查问题，我们掌握了 Linux 环境下的调试思维，先定位问题场景（工具 / 代码 / 环境），再拆解依赖关系。这种闭环，不仅解决了当前实验的障碍，更为后续复杂 OS 实验奠定了基础。

MINIBU